

Laboratory 2: Agents

Constructing a Goal-Based Agent using a Large Language Model

Eshan Saikia
2023A7PS1024G

The accompanying `agents.ipynb` contains the executable implementation and validation checks. This report records the problem formulation, agent design, LLM-assisted implementation, measured results, and reflections.

1 Problem Specification and Design

The warehouse is represented by the supplied two-dimensional grid:

```
#####  
#S...#.....G#  
#.#...#####.#  
#...##.....#  
#.#####.###.#.###.#  
#.....#.....#  
#####
```

Here, S is the starting position, G is the goal, $\#$ is an obstacle, and a period is a free cell. The vehicle may move Up, Down, Left, or Right, changing its position by one grid square per action.

Environment. The static warehouse map and obstacle layout.

Goal. Reach the grid cell containing G .

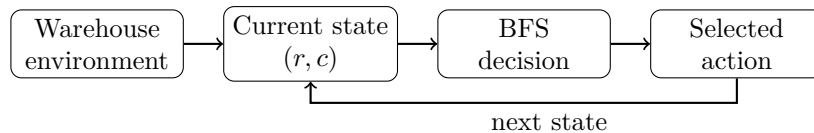
State. The vehicle's current position (r, c) . The search additionally stores frontier and parent information for exploration and path reconstruction.

Actions. Up, Down, Left, Right, provided the resulting position remains inside the grid and is not an obstacle.

Decision component. Breadth-first search (BFS).

The agent is goal-based because it has an explicit objective and selects actions with respect to possible future states leading to that objective. A simple reflex agent would react only to the present state rather than search for an action sequence that reaches a specified goal.

BFS is appropriate because the environment is finite and deterministic and every movement has cost 1. It explores states in increasing path depth and therefore returns a minimum-move solution when one exists.



2 LLM-Assisted Implementation

The implementation request supplied to the LLM was:

I am implementing the warehouse navigation problem below as a goal-based agent in Python. Represent the warehouse as a two-dimensional grid. The agent starts at **S** and must reach **G**. **#** cells are obstacles, **.** cells are free, and the only actions are Up, Down, Left, and Right; each move changes the position by one cell. Implement a simple, deterministic search-based goal-directed agent that finds a collision-free path, prints the path and its length, and reports failure if no path exists. Store parent/action information so the path can be reconstructed. Explain the chosen search method and why it is appropriate for this environment. Keep the implementation dependency-free and easy to inspect.

The first generated implementation worked on the supplied warehouse. During verification, explicit checks were added for legal movement, obstacle avoidance, start/goal handling, and path reconstruction, together with path visualisation. These additions do not alter the problem or search method.

The source was inspected before execution to verify the state representation, legal successor generation, FIFO frontier, duplicate prevention, path reconstruction, and goal test.

3 Experimental Results

3.1 Main Warehouse Run

The executed agent found a collision-free path from S to G .

Measure	Result
Path found	Yes
Path length	20 moves
States expanded	59

The returned action sequence was:

Right	Right	Right	Down	Right
Right	Right	Right	Up	Right
Right	Right	Right	Right	Right
Right	Right	Right	Right	Right

The notebook independently checks that the path starts at S , ends at G , uses only legal adjacent moves, and never enters an obstacle. All checks passed.

3.2 Prompt Engineering Questions

Did the LLM generate a working program on the first attempt? Yes. The first generated implementation produced a valid path for the supplied warehouse when executed.

How could the prompt be improved? It could explicitly specify the state representation, legal actions, collision handling, goal test, parent tracking, path reconstruction, and failure behaviour. This reduces ambiguity while leaving implementation details open.

What search algorithm did the LLM choose? BFS.

Why was it selected? The environment is finite and deterministic with equal movement costs, making BFS simple to inspect and guaranteeing a shortest path in number of moves for this setting.

4 Reflection Questions

1. **Why formulate the problem before writing the algorithm?** The formulation fixes the state, actions, transitions, initial state, goal, and objective. It provides the specification against which the generated implementation can be checked.
2. **Why is this a goal-based agent?** Its behaviour is directed by the explicit goal of reaching G . It searches possible future states and selects actions as part of a sequence that achieves that goal, rather than reacting only to the current cell.
3. **What did the LLM contribute?** It translated the specified agent design into executable Python, reducing implementation effort. Human verification was still required to establish that the code satisfied the specification and produced a valid path.
4. **What could go wrong without testing?** Generated code may run while mishandling obstacles, state transitions, goal recognition, duplicate states, or path reconstruction. Tests with known expected behaviour are therefore necessary.

5 Think About It

If the warehouse becomes twice as large, the same BFS formulation remains correct for the same unit-cost movement model, but the number of reachable states and the memory required by the frontier can increase substantially. For a larger environment, an informed search such as A* could reduce unnecessary exploration by using information about the remaining distance to the goal while retaining the same state and action definitions.